



23 FÉVRIER 2026

DOCUMENTATION ARCHITECTURE

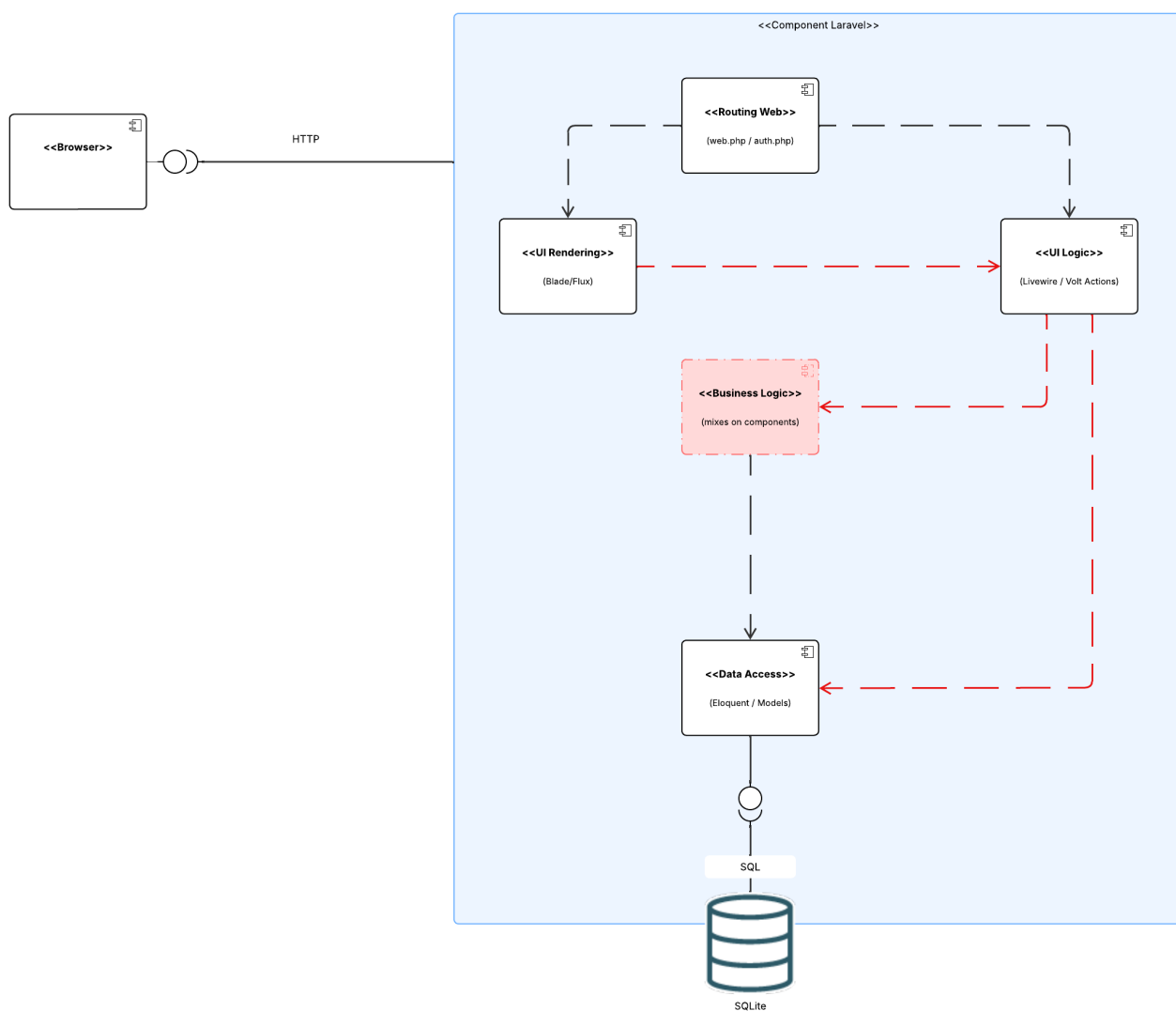
LAURENT GOUROUVIN

1. Introduction	2
2. Architecture de base	2
A. Avantage de l'architecture livewire	3
B. Inconvénients de l'architecture livewire	3
C. Composants de l'architecture livewire	4
D. Chemin d'une action utilisateur - Création de note	4
3. Architecture cible	5
A. Composant d'architecture	5
Back-end	6
Front-end	6
Communication	6
B. Composants de la nouvelle architecture	7
C. Chemin d'une action utilisateur - Création de note	7
D. Spécification API REST	8
Auth	8
Password	8
Profile	9
Note	9
Tag	9
Email	9
Exemple d'api	10
E. Data flow state management	10
4. Analyse de l'écart	11
Composants modifiés ou supprimés	11
Composants ajoutés	11
5. Justification de l'approche architecturale	12

1. INTRODUCTION

Renote a été migrée d'un monolithe server-driven (Blade/Livewire) vers une architecture client-driven avec React, via un découplage complet front/back. Le back a été refactorisé en MVC/SOLID avec une API REST stable, et le front s'appuie désormais sur un state management pour centraliser l'état, les actions et les appels API.

2. ARCHITECTURE DE BASE



A. AVANTAGE DE L'ARCHITECTURE LIVEWIRE

L'architecture initiale, utilisant Livewire propose plusieurs avantages :

- **Simplicité de développement** : Pas de séparation front/back nécessaire, un seul langage (PHP), déploiement simplifié.
- **Réactivité sans JavaScript** : Livewire permet des interfaces réactives sans écrire de JS complexe, idéal pour des équipes PHP.
- **SEO natif** : Le rendu côté serveur génère du HTML complet, favorable au référencement naturel.
- **Idéal pour les petites applications** : Pour une application simple avec peu d'interactions complexes, cette architecture est suffisante et rapide à mettre en place.
- **Cohérence de stack** : Tout reste dans l'écosystème Laravel – validations, sessions, middleware – sans couche supplémentaire.

B. INCONVÉNIENTS DE L'ARCHITECTURE LIVEWIRE

L'architecture initiale expose plusieurs inconvénients :

- **Couplage fort** : L'UI, la logique métier et l'accès aux données sont mélangés dans les composants Livewire. Un composant peut effectuer simultanément du traitement métier, de la validation, de la persistance et du rendu UI, ce qui viole le principe de responsabilité unique.
- **Absence d'API REST** : Les routes reposent exclusivement sur des routes Web liées à Livewire, rendant l'application inaccessible depuis un client externe (mobile, SPA, service tiers).
- **Mélange des responsabilités** : Sans couche service dédiée, la logique métier est dupliquée ou dispersée entre les composants, rendant le code difficile à maintenir.

C. COMPOSANTS DE L'ARCHITECTURE LIVEWIRE

Composant Livewire	Rôle
Auth/login.blade.php	Formulaire de connexion
Auth/register.blade.php	Formulaire d'inscription
Auth/forgot-password.php	Envoi lien reset password
Auth/reset-password.blade.php	Reset du mot de passe
Auth/verify-email.blade.php	Vérification email
Auth/confirm-password.blade.php	Confirmation du mot de passe
notes.blade.php	Liste et création de notes
Tag-form.blade.php	Création de tags
Settings/profile.blade.php	Mise à jour du profil
Settings/password.blade.php	Mise à jour de mot de passe

D. CHEMIN D'UNE ACTION UTILISATEUR - CRÉATION DE NOTE

Composants traversés :

1. **notes.blade.php** : L'utilisateur saisit le texte et sélectionne un tag, clique sur « Add note »
2. **Livewire\Notes** (composant Volt) : La méthode `save()` est déclenchée via `wire:submit`
3. **NoteService::createNote()** : Crée la note via Eloquent
4. **Note (Eloquent Model)** : Permet de persister la donnée en base de donnée
5. Livewire re-render automatiquement le composant avec les nouvelles données

Données échangées :

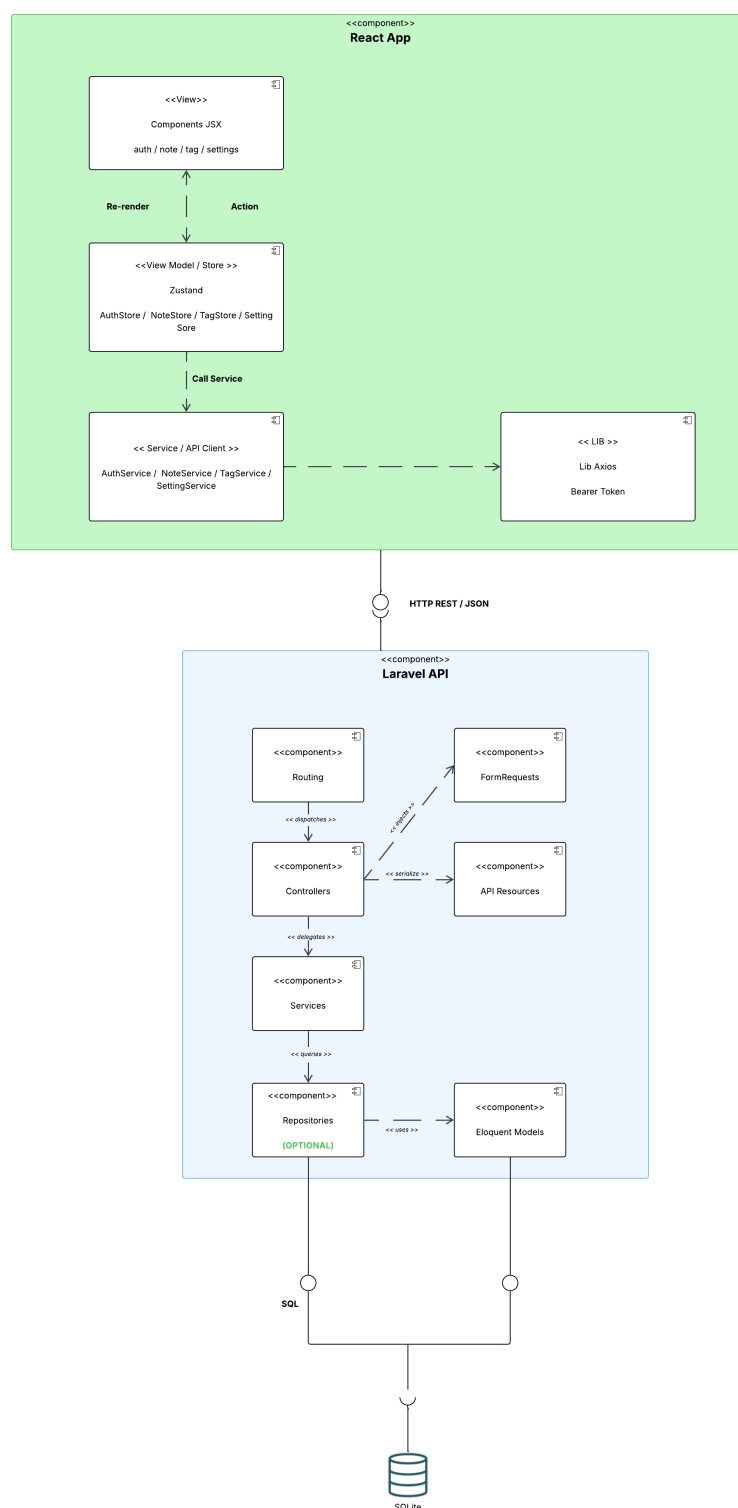
Requête HTTP POST (websocket Livewire) → { text: "Ma note", tag_id: 1 }

Réponse → HTML re-rendu par le serveur

Ajouter un diagramme ICI ?

3. ARCHITECTURE CIBLE

A. COMPOSANT D'ARCHITECTURE



BACK-END

- **Controllers API** : Reçoivent les requêtes HTTP, délèguent aux services, retournent des réponses JSON standardisées
- **Services** : Contiennent la logique métier, injectés via leurs interfaces (IoC)
- **API Resources** : Transforment les modèles Eloquent en JSON, évitent l'exposition de données sensibles
- **Form Requests** : Valident et autorisent les requêtes en amont des controllers
- **Laravel Sanctum** : Gère l'authentification via tokens Bearer
- **Eloquent ORM** : Driver d'accès à la base de données SQLite via PDO

FRONT-END

- **React** : Rendu UI et navigation client-side
- **Zustand** : State management centralisé
- **Axios** : Client HTTP pour les appels API

COMMUNICATION

- Protocole **HTTP/REST** entre **React** et **Laravel**
- Format **JSON** pour tous les échanges
- Authentification par **token Bearer** (Sanctum)

B. COMPOSANTS DE LA NOUVELLE ARCHITECTURE

Composant Livewire	Rôle	React
Auth/login.blade.php	Formulaire de connexion	LoginForm.jsx
Auth/register.blade.php	Formulaire d'inscription	RegisterForm.jsx
Auth/forgot-password.php	Envoi lien reset password	ForgotPasswordForm.jsx
Auth/reset-password.blade.php	Reset du mot de passe	ResetPasswordForm.jsx
Auth/verify-email.blade.php	Vérification email	VerifyEmail.jsx
Auth/confirm-password.blade.php	Confirmation du mot de passe	ConfirmPassword.jsx
notes.blade.php	Liste et création de notes	NoteList.jsx + NoteForm.jsx
Tag-form.blade.php	Création de tags	TagForm.jsx
Settings/profile.blade.php	Mise à jour du profil	ProfileForm.jsx
Settings/password.blade.php	Mise à jour de mot de passe	PasswordForm.jsx

C. CHEMIN D'UNE ACTION UTILISATEUR - CRÉATION DE NOTE

Composants traversés :

1. **NoteForm.jsx** : l'utilisateur saisit le texte et sélectionne un tag, clique sur « Add note »
2. **useNoteStore** (Zustand) : l'actions addNote(text, tagId) est appelée
3. **noteService.js** : createNote() envoie POST /api/note via Axios avec le token Bearer
4. **Routes/api.php** : Laravel reçoit la requête et la dirige vers le NoteController
5. **NoteCreateRequest** : Valide les données (text required, tag_id exists)

6. **NoteController::createNote()** : Délègue au service
7. **NoteService::createNoteApi()** : Crée la note via Eloquent
8. **Note (Eloquent Model)** : persiste la note en base de donnée
9. **NoteResource** : Transforme le modèle en JSON
10. Retour JSON -> noteStore met à jour notes[] -> NoteList.jsx se re-render

Données échangées :

Request → { text: "Ma note", tag_id: 1 }

Response → { status: "success", message: "Note created", data: { id: 15, text: "Ma note", tag: { id: 1, name: "LoL" }, created_at: "..." } }

D. SPÉCIFICATION API REST

AUTH

Méthode	URL	Description	Protégée
POST	/api/auth/login	Se connecter	Non
POST	/api/auth/register	Créer un compte	Non
POST	/api/auth/logout	Se déconnecter	Oui

PASSWORD

Méthode	URL	Description	Protégée
POST	/api/password/forgot-password	Envoi lien de reset	Non
POST	/api/password/reset	Reset du mot de passe	Non
POST	/api/password/confirm	Confirmation du mot de passe	Oui
PATCH	/api/password/update	Mise à jour du mot de passe	Oui

PROFILE

Méthode	URL	Description	Protégée
GET	/api/profile/me	Récupération du profil	Oui
PUT	/api/profile	Mise à jour du profil	Oui
DELETE	/api/profile	Suppression du profil	Oui

NOTE

Méthode	URL	Description	Protégée
GET	/api/note	Liste des notes	Oui
POST	/api/note	Création d'une note	Oui
DELETE	/api/note/{id}	Suppression d'une note	Oui

TAG

Méthode	URL	Description	Protégée
GET	/api/tag	Liste des tags	Oui
POST	/api/tag	Création d'un tag	Oui

EMAIL

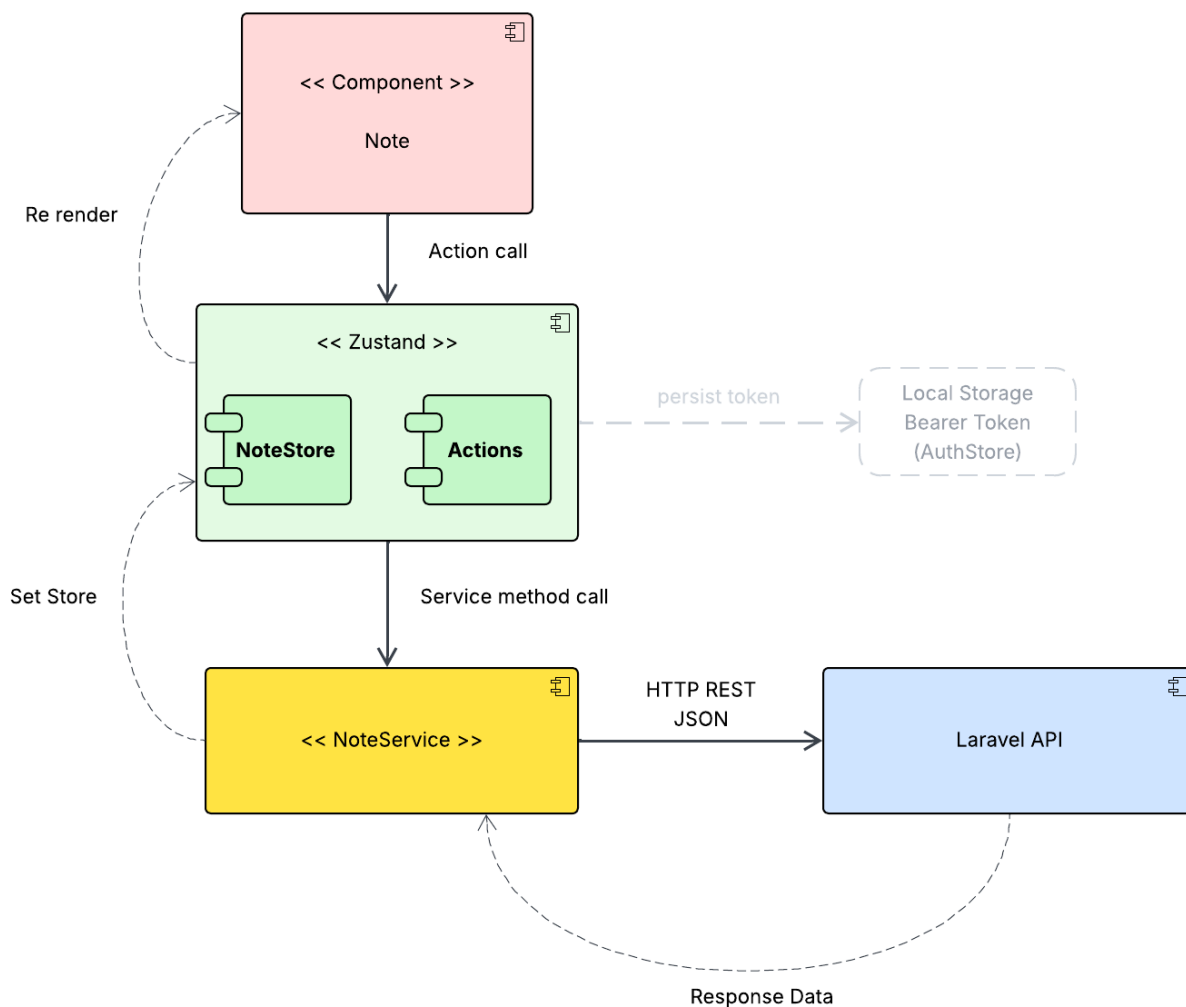
Méthode	URL	Description	Protégée
POST	/api/email/verify/send	Envoi email de vérification	Oui
GET	/api/email/verify/{id}/{hash}? expires=&signature	Vérification de l'email	Oui

EXEMPLE D'API

```
1 // POST /api/auth/login
2 // Request
3 { "email": "user@example.com", "password": "password" }
4 // Response 200
5 {
6   "status": "success",
7   "message": "Login successful",
8   "data":
9     {
10      "user":
11        {
12          "id": 1,
13          "name": "John",
14          "email": "user@example.com"
15        },
16      "token": "1|abc123..."
17    }
18 }
19 // Response 401 - Invalid credentials
20 // Response 422 - Validation failed
```

E. DATA FLOW STATE MANAGEMENT

- **Action utilisateur** : ex: clic sur "Créer une note »
- **Zustand action** : *createNote(text, tagId)* appelée depuis le composant
- **Effet API** : Axios envoie *POST /api/note* avec le token Bearer
- **Réponse Laravel** : JSON { status, message, data }
- **Update store** : La note est ajoutée dans *notes[]* du store Zustand
- **Re-render React** : Les composants abonnés au store se mettent à jour automatiquement



4. ANALYSE DE L'ÉCART

COMPOSANTS MODIFIÉS OU SUPPRIMÉS

- Les composants Livewire/Volt sont remplacés par des composants React la logique UI et le rendu migrent côté client.
- Le routing Web Laravel est remplacé par un routing API REST pur
- Les vues Blade ne servent plus qu'à servir le point d'entrée React

COMPOSANTS AJOUTÉS

- Application React avec routing client-side

- Store Zustand pour la gestion d'état
- Controllers API dédiés (AuthController, NoteController, TagController, ProfileController, PasswordController, EmailController)
- API Resources (UserResource, NoteResource, TagResource)
- Form Requests pour la validation des entrées API
- Laravel Sanctum pour l'authentification par token

5. JUSTIFICATION DE L'APPROCHE ARCHITECTURALE

Laravel Sanctum : Choisi pour sa simplicité d'intégration avec Laravel et sa gestion native des tokens API. Adapté pour une SPA qui consomme sa propre API.

Zustand : Préféré à Redux pour sa légèreté et sa syntaxe minimaliste. Pour une application de la taille de Renote, Redux introduirait une complexité inutile (boilerplate, actions, reducers). Zustand permet de définir un store en quelques lignes tout en restant scalable.

SOLID : Les interfaces (AuthServiceInterface, NotesServiceInterface, etc.) permettent d'inverser les dépendances (DIP) et de rendre chaque composant remplaçable sans modifier le reste du code (OCP). Les services ont une responsabilité unique et ne connaissent pas la couche HTTP (SRP).

API Resources : Garantissent qu'aucune donnée sensible n'est exposée accidentellement. Elles constituent un contrat stable entre le back et le futur front React.

Format de réponse standardisé : Le trait ApiResponse centralise le format { status, message, data} sur tous les endpoints, facilitant la gestion des réponses côté Axios/Zustand.